

susi-bdc

Pipeline de saisie des bons de commande papier Atlas For Men dans le CRM SUSI (« Commandes Rapides ») : lecture vision des scans par sous-agents Claude, revue humaine avec score de confiance, saisie pilotée dans SUSI, vérification post-saisie. Deux frontaux — une plateforme web et Claude Code — partagent le même état sur disque.

CODE

/Users/djisse/susi-bdc (Mac de Djibril)

PLATEFORME

FastAPI · port 8760 · un seul fichier app/app.py

AGENTS

~/.claude/agents/ + skill ~/.claude/skills/susi-saisie-bdc/

ÉTAT

extraction + revue validées saisie SUSI jamais exécutée

1. Le métier en deux minutes

Atlas For Men reçoit des commandes papier : un **BDC** (bon de commande manuscrit) accompagné le plus souvent d'un **chèque**. Les liasses sont scannées en PDF — nom de fichier normé, ex. 28082026FR_ATLAS_III_FID_50CH1.pdf : date, pays, FID (fidélisation, n° client présent) ou RECRUT (nouveau client), 50CH = 50 commandes par chèque (OA = sans paiement joint), liasse n° 1. Dans un lot CH, **pages impaires = chèques, pages paires = BDC**.

La saisie se fait dans SUSI, module « Saisie commande rapide » (app web interne, clavier : F5 créer un lot, TAB entre champs, F4 bloc suivant, Echap clôturer). La référence fonctionnelle est le manuel Mada, converti en texte : manuel-commandes-rapides.md à la racine du projet — **à lire en premier**, tout le pipeline en découle (y compris les cas particuliers p. 16 : deux chèques, chèque absent → OA, contre-remboursement → OA, espèces → ne pas saisir).

2. Architecture



Le dossier du lot est la seule source de vérité : la plateforme web et une session Claude Code y lisent et écrivent les mêmes fichiers, donc un lot commencé d'un côté se poursuit de l'autre. Seuls `susi-saisisseur` et `susi-verificateur` touchent SUSI, à travers le Chrome de l'utilisateur connecté au VPN — jamais en parallèle.

LES COMPOSANTS

COMPOSANT	FICHIER	RÔLE
Skill orchestratrice	<code>~/ .claude/skills/susi-saisie-bdc/SKILL.md</code>	Le pipeline en 7 phases, utilisable en session Claude Code. Les prompts des jobs de la plateforme y renvoient : c'est la spec de référence.
bdc-lecteur	<code>~/ .claude/agents/bdc-lecteur.md</code>	Lecture vision d'une paire chèque+BDC (l'OCR des scans est inutilisable). ~5 commandes par exemplaire, jusqu'à 10 en parallèle. Zoom 300 dpi via <code>pdftoppm -x -y -W -H</code> sur les zones douteuses. Sort un <code>extraits/cmd_NN.json</code> par commande.
susi-saisisseur	<code>~/ .claude/agents/susi-saisisseur.md</code>	Saisit le lot validé dans SUSI (F5/F4/Echap selon le manuel) via les outils claude-in-chrome. Journal <code>saisie.log.json</code> pour reprise. Un seul exemplaire, jamais en parallèle. Ne saisit jamais d'identifiants.
susi-verificateur	<code>~/ .claude/agents/susi-verificateur.md</code>	Post-saisie, lecture seule : compare SUSI (rapport Excel + détail UI) au scan d'origine , score 0-100 par commande, sort <code>verification.json</code> + <code>review.md</code> triés par score croissant.
Plateforme web	<code>susi-bdc/app/app.py</code>	Upload, jobs, écran de revue (scores pré-saisie calculés côté app, tout éditable), validation, écran de vérification. Auth HTTP Basic (schéma TLMC) : <code>SUSI_BDC_UTILISATEURS=nom:mdp:role</code> dans <code>app/.env</code> , rôles <code>admin</code> / <code>viewer</code> .

ANATOMIE D'UN LOT

```
lots/28082026FR_ATLAS_III_FID_50CH1/
├─ *.pdf                # scan source (copié à l'upload)
├─ meta.json            # pays, type FID/RECRUT, paiement CH/OA, nb commandes – déduits du nom
├─ pages/p-001.jpg ...  # rendu 150 dpi, padding 3 chiffres normalisé
├─ extraits/cmd_NN.json  # extraction + bloc "revue" {statut, verifie, par, commentaire}
├─ lot.json              # consolidation, puis version validée
├─ recap.md             # tableau de contrôle phase 3
├─ validation.json      # checkpoint humain – la saisie est refusée sans lui
├─ saisie.log.json / saisie_resultat.md
├─ verification.json / review.md
└─ jobs/{extraction,saisie,verification}.{log,status.json}
```

La phase affichée dans l'UI est *entièrement déduite* de ces fichiers (`etat_lot()` dans app.py) — il n'y a aucune base de données.

LE SCORE DE CONFIANCE

Deux scores distincts, même échelle (≥ 90 OK 70-89 à vérifier < 70 review obligatoire) :

- **Pré-saisie** (revue) — calculé par `score_extraction()` dans app.py : 100 moins des retenues (arithmétique lignes+frais ≠ total –40, chèque ≠ total –40, refco illisible –25, confiance basse/moyenne par article –20/–10, anomalies –5 chacune plafonné à –20).
- **Post-saisie** (vérification) — calculé par l'agent `susi-verificateur` avec son propre barème (écart montant –40, article faux –25, mauvais client –30, absente/en double = 0), en retournant *toujours* au scan quand SUSI et l'extraction divergent.

3. État des lieux

BRIQUE	STATUT	PREUVE
Upload + rendu + méta	testé	lots 2CH9 (mini) et 50CH1 (réel)
Extraction via plateforme	testé	50/50 commandes en 16 min, 10 lecteurs en parallèle, total 2 396,58 € équilibré, doublon de scan détecté (cmd 26 à écarter)
Revue / édition / validation	testé	édition d'articles, statuts, validation → lot.json
Log des jobs en direct	testé	stream-json parsé en lignes lisibles, rafraîchi toutes les 5 s
Saisie SUSI réelle	jamais exécutée	l'agent n'a jamais vu l'UI Commandes Rapides — première fois à faire supervisée
Vérification post-saisie	jamais exécutée	dépend de la première saisie

Le lot réel 50CH1 est extrait et en attente de revue. Ne lancez pas sa saisie sans Djibril : elle crée de vraies commandes et de vrais encaissements. La cmd 26 est un double passage scanner (même chèque CMC7, même BDC que la paire voisine) — elle doit rester « écartée ».

4. Démarrer et tester

PRÉREQUIS (SUR LA MACHINE QUI HÉBERGE)

1. **macOS avec le dépôt** `/Users/djisse/susi-bdc` + les fichiers `~/ .claude/agents/*` et `~/ .claude/skills/susi-saisie-bdc/` (à copier si vous reprenez sur une autre machine).
2. `pdftoppm` (poppler) : `brew install poppler`.
3. **CLI Claude Code connecté** : `claude /login` (sans sudo). Testez : `claude -p "ok" --output-format text`.
4. Pour la saisie/vérification uniquement : VPN AFM actif + session SUSI ouverte dans le Chrome de la machine (login utilisateur `AA_AFM_100` — les agents ne saisissent *jamais* de mot de passe), et l'intégration Chrome de Claude (outils `claude-in-chrome`).

LANCER

```
/Users/djisse/susi-bdc/app/start.sh    # sert sur http://0.0.0.0:8760
/Users/djisse/susi-bdc/app/tunnel.sh    # URL publique https (Cloudflare, éphémère)
```

Comptes dans `app/.env` (non versionné, demander à Djibril ou en générer) :

`SUSI_BDC_UTILISATEURS=nom:mdp:admin,nom2:mdp2:viewer`. Un compte admin est auto-généré au premier démarrage et affiché en console.

PARCOURS DE TEST (10 MINUTES, SANS TOUCHER SUSI)

1. Ouvrir la plateforme → le lot `29082026FR_ATLAS_III_FID_2CH9` (mini-lot de test : 4 pages, 2 commandes) est déjà là. Pour repartir de zéro : supprimer `extraits/*`, `lot.json`, `recap.md`, `jobs/*` du dossier.
2. « Lancer l'extraction » → suivre le log en direct (~5-7 min : session, sous-agents, consolidation).
3. Ouvrir la revue : 2 commandes avec scores et anomalies (chèques de tiers, case paiement non cochée...). Ouvrir une commande : scans en face du formulaire, modifier, « Enregistrer ».
4. « Valider le lot » → `validation.json` apparaît, l'étape Saisie se déverrouille. **S'arrêter là** tant que la première saisie supervisée n'a pas eu lieu.

Équivalent en session Claude Code : glisser un PDF de lot et dire « traite mes bdc » (la skill `susi-saisie-bdc` se déclenche) — même état sur disque, même revue possible ensuite via l'app.

5. Les pièges déjà payés

Chacun de ces points a coûté un débogage réel — ne les réapprenez pas :

- `stdin=subprocess.DEVNULL` **est obligatoire** sur le `claude -p` du job runner : sinon il hérite du stdin d'uvicorn et se fige indéfiniment (constaté : 3 h à 0 % CPU, log vide).
- **Épurer l'environnement** avant de lancer `claude -p` depuis un process lancé par une session Claude (regex `CLAUDE|ANTHROPIC|BAGGAGE|AI_AGENT|SENTRY`) : les variables héritées court-circuitent l'auth Trousseau → 401 « OAuth access token is invalid » même connecté.

- `--mcp-config` exige `{"mcpServers": {}}`, pas `{}` (erreur « Invalid MCP configuration »). L'extraction tourne avec `--strict-mcp-config` (aucun MCP → démarrage rapide) ; la saisie garde la config par défaut (il lui faut `claude-in-chrome`).
- `--output-format stream-json --verbose` + parsing ligne à ligne (`_lignes_lisibles()`) pour un log visible en direct — en mode `text`, rien ne sort avant la fin.
- `pdftoppm` adapte le zéro-padding au nombre de pages (`p-1.jpg` pour un petit PDF) : l'upload renomme systématiquement en `p-NNN.jpg`, sinon le tri lexicographique casse.
- **SUSI est fragile** (mesuré sur `susi-rapports` : ~80-100 requêtes puis HTTP 500, sessions qui expirent sous charge) : un seul job à la fois (verrou global), saisie strictement séquentielle, jamais deux agents sur SUSI en parallèle.
- **Métier** : sur un chèque, les *lettres font foi* sur les chiffres ; sur un BDC, recalculer depuis les lignes. Attention aux doublons de scan (vérifier la ligne CMC7), aux confusions manuscrites 0/O, 1/1, 2/2, 5/5, B/8 sur les refco, et aux scans à 180°.

6. Chantiers, par priorité

1. **Première saisie SUSI supervisée** (avec Djibril, VPN + session ouverte) : roder `susi-saisisseur` sur l'UI réelle — les raccourcis F5/F4/Echap viennent du manuel, pas encore confrontés au terrain. Puis premier run de `susi-verificateur`.
2. **Validation des refco au catalogue** : croiser chaque refco extraite avec le référentiel (`atlasformen.fr` ou export SUSI) dès l'extraction — l'agent l'a fait ponctuellement (KB179 → K8179), à systématiser dans la phase 3 ou dans l'app.
3. **Détection de doublons systématique** : comparer les lignes CMC7 et les n° clients entre commandes d'un lot (le doublon cmd 26 a été attrapé par l'agent ; un contrôle déterministe dans `score_extraction()` /consolidation serait plus sûr).
4. **Durcir la plateforme** : comptes nominatifs par membre de l'équipe, tunnel stable (Cloudflare nommé ou Tailscale au lieu du trycloudflare éphémère), file de jobs persistante (le verrou est en mémoire : un restart pendant un job laisse un `en_cours` orphelin — le statut se répare en relançant le job).
5. **Tests** : rien n'est couvert ; commencer par `score_extraction()`, le parsing de nom de lot (`RX_NOM`) et `_lignes_lisibles()`.

7. Références

- Manuel métier : `susi-bdc/manuel-commandes-rapides.md` (source : PPTX « Manuel d'utilisation Commandes Rapides Mada »)
- Spec pipeline : `~/claude/skills/susi-saisie-bdc/SKILL.md` · README : `susi-bdc/README.md`
- SUSI : `http://10.210.0.20/AFM/APP/SUSI/` (VPN) — capacité serveur documentée dans `~/claude/agents/susi-rapports.md`
- Projet frère (auth, patterns) : plateforme TLMC `/Users/djisse/tlmc-remise`
- Lot réel en cours : `lots/28082026FR_ATLAS_III_FID_50CH1` — 50 extraits, 2 396,58 €, en attente de revue ; mini-lot de test : `lots/29082026FR_ATLAS_III_FID_2CH9`

Passation rédigée le 03/09/2026. L'état « jamais exécuté » de la saisie/vérification est celui de cette date — mettez cette page à jour après la première saisie réelle.